

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**



ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И УПРАВЛЯЮЩИХ СИСТЕМ

Лабораторная работа №2

по дисциплине: **Операционные системы**
тема: **«Виртуальная память в ОС Linux»**

Выполнил: ст. группы ПВ-233
Мовчан Антон Юрьевич

Проверили:
пр. Четвертухин Виктор Романович

Белгород 2025 г.

Лабораторная работа №2

Виртуальная память в ОС Linux

Вариант 8

Цель работы: Изучить концепцию виртуальной памяти в Linux, механизмы отображения виртуальных адресов в физические, организацию памяти процесса, работу подкачки и кэширования.

1. Задание «Бережливый сборщик и гигантский массив». Подготовить объект разделяемой памяти (`int fd = shm_open(name, O_CREAT|O_RDWR, 0600);`) размером M (M должен быть кратным размеру страницы памяти, причем $M < 0.5 \times \text{ОЗУ}$, чтобы позволить ОС работать). Заполнить объект случайными целыми числами (`uint64_t`). Реализовать на языке C программу

для вычисления суммы всех элементов объекта (сложение ведётся в арифметике по модулю 2^{64} — при переполнении старшие биты автоматически отбрасываются, аккумулятор имеет тип `uint64_t`). Программа должна использовать два экспериментальных алгоритма: а) алгоритм А (блочно-буферизированное чтение) — читать массив с помощью `read()` / `pread()` фиксированными блоками размера K , последовательно обрабатывать содержимое буфера, а затем перезаполнять буфер; память под буфер — одна, переиспользуемая; б) алгоритм В — с помощью `mmap()` отобразить объект окнами размера W , а потом двигать «скользящее окно» (это фиксированный по размеру участок памяти, который последовательно отображается и «перекачивается» по данным, переиспользуя ту же область памяти для нового фрагмента). Подобрать параметры M , K , W для своей конфигурации, обеспечивающие наилучший баланс между пропускной способностью и общими затратами системного времени. Отразить в отчете затраты на страничные преобразования и нагрузку на память через: `dTLB-loads` — число обращений к TLB данных (Translation Lookaside Buffer) при загрузках (чтении данных по виртуальным адресам) и `dTLB-load-misses` — число промахов TLB, когда процессор не нашёл нужное отображение и пришлось обращаться к таблице страниц. Можно использовать такую команду:

```
sudo perf stat -e dTLB-loads, dTLB-load-misses ./lab2_c.o
```

Код программы:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>
#include <time.h>
#include <string.h>
#include <inttypes.h>

#define SHM_NAME "/my_shm"
#define M (4ULL << 30) // 4 ГБ
#define K (128 * 1024) // 128 КБ
#define W (128 * 1024 * 1024) // 128 МБ

// Функция для заполнения объекта случайными числами
void fill_shm(int fd) {
    uint64_t *buffer = malloc(K);
    if (!buffer) {
        perror("malloc failed");
        exit(1);
    }
    srand(time(NULL));
    off_t offset = 0;
    while (offset < M) {
        for (size_t i = 0; i < K / sizeof(uint64_t); i++) {
            buffer[i] = ((uint64_t)rand() << 32) | rand();
        }
        offset += K;
    }
}
```

```

    ssize_t written = write(fd, buffer, K);
    if (written < 0) {
        perror("write failed");
        free(buffer);
        exit(1);
    }
    offset += written;
}
free(buffer);
}

```

// Алгоритм А: блочно-буферизированное чтение

```

uint64_t algorithm_a(int fd) {
    uint64_t sum = 0;
    uint64_t *buffer = malloc(K);
    if (!buffer) {
        perror("malloc failed");
        exit(1);
    }
    off_t offset = 0;
    while (offset < M) {
        ssize_t bytes_read = pread(fd, buffer, K, offset);
        if (bytes_read < 0) {
            perror("pread failed");
            free(buffer);
            exit(1);
        }
        for (size_t i = 0; i < bytes_read / sizeof(uint64_t); i++) {
            sum += buffer[i]; // Сумма по модулю 2^64
        }
        offset += bytes_read;
    }
    free(buffer);
    return sum;
}

```

// Алгоритм В: скользящее окно с mmap

```

uint64_t algorithm_b(int fd) {
    uint64_t sum = 0;
    void *addr = NULL;
    off_t offset = 0;
    while (offset < M) {
        size_t len = (offset + W <= M) ? W : M - offset;
        addr = mmap(NULL, len, PROT_READ, MAP_SHARED, fd, offset);
        if (addr == MAP_FAILED) {
            perror("mmap failed");
            exit(1);
        }
        uint64_t *data = (uint64_t *)addr;
        for (size_t i = 0; i < len / sizeof(uint64_t); i++) {
            sum += data[i]; // Сумма по модулю 2^64
        }
        munmap(addr, len);
        offset += len;
    }
    return sum;
}

```

```

int main() {
    // Создание объекта разделяемой памяти
    int fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0600);
    if (fd < 0) {
        perror("shm_open failed");
        exit(1);
    }
    if (ftruncate(fd, M) < 0) {
        perror("ftruncate failed");
    }
}

```

```

    exit(1);
}

// Заполнение объекта случайными числами
printf("Filling shared memory...\n");
fill_shm(fd);

// Алгоритм A
printf("Running Algorithm A...\n");
struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);
uint64_t sum_a = algorithm_a(fd);
clock_gettime(CLOCK_MONOTONIC, &end);
double time_a = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
printf("Algorithm A: Sum = %" PRIu64 " , Time = %.6f seconds\n", sum_a, time_a);

// Алгоритм B
printf("Running Algorithm B...\n");
clock_gettime(CLOCK_MONOTONIC, &start);
uint64_t sum_b = algorithm_b(fd);
clock_gettime(CLOCK_MONOTONIC, &end);
double time_b = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
printf("Algorithm B: Sum = %" PRIu64 " , Time = %.6f seconds\n", sum_b, time_b);

// Очистка
shm_unlink(SHM_NAME);
close(fd);
return 0;
}

```

Вывод программы:

```

anton@Anton:~/Study/Projects-Documentation-Toolkit/Операционные системы/Lab2/src$ sudo ./task
Filling shared memory...
Running Algorithm A...
Algorithm A: Sum = 7817401526606795685, Time = 0.657996 seconds
Running Algorithm B...
Algorithm B: Sum = 7817401526606795685, Time = 0.662357 seconds
anton@Anton:~/Study/Projects-Documentation-Toolkit/Операционные системы/Lab2/src$ sudo perf stat -e dTLB-loads,dTLB-load-misses ./task
Filling shared memory...
Running Algorithm A...
Algorithm A: Sum = 16999033362742161794, Time = 0.661003 seconds
Running Algorithm B...
Algorithm B: Sum = 16999033362742161794, Time = 0.647105 seconds

Performance counter stats for './task':

      4,843,194      dTLB-loads
      857,053      dTLB-load-misses           #   17.70% of all dTLB cache accesses

      7.485241230 seconds time elapsed

      5.634482000 seconds user
      1.846158000 seconds sys

```

Вывод: в ходе выполнения л.р я изучил концепцию виртуальной памяти в Linux, механизмы отображения виртуальных адресов в физические, организацию памяти процесса, работу подкачки и кэширования.